National University of Singapore

CS1101S — Programming Methodology

AY2023/2024 Semester 1

Final Assessment

Time allowed: 2 hours

QUESTION PAPER

INSTRUCTIONS

1. This **QUESTION PAPER** contains **28** Questions in **9** Sections, and comprises **18** printed pages, including this page.
2. The **ANSWER SHEET** comprises **4** printed pages.
3. Use a pen or pencil to **write** your **Student Number** in the designated space on the front page of the **ANSWER SHEET**, and **shade** the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME!
4. You must **submit only** the **ANSWER SHEET** and no other documents. Do not tear off any pages from the ANSWER SHEET.
5. All questions must be answered in the space provided in the **ANSWER SHEET**; no extra sheets will be accepted as answers.
6. Write legibly with a **pen** or **pencil** (do not use red color). Untidiness will be penalized.
7. For **multiple choice questions (MCQ)**, **shade** in the **circle** of the correct answer **completely**.
8. The full score of this assessment is **100** marks.
9. This is a **Closed-Book** assessment, but you are allowed to bring with you one double-sided **A4 / letter-sized sheet** of handwritten or printed **notes**.
10. Where programs are required, write them in the **Source §4** language. A **reference** of some of the **pre-declared functions** is given in the **Appendix** of the Question Paper.
11. In any question, unless it is specifically allowed, your answer **must not use functions** given in, or written by you for, other questions.

Section A: Orders of Growth [8 marks]

(1) [3 marks] (MCQ)

What is the order of growth of the **space** needed to apply the function `dat` to argument *n*?

```
function dat(n) {
    const A = [];
    for (let i = 0; i <= n; i = i + 1) {
        A[i] = [];
    }
    for (let i = 0; i <= n; i = i + 1) {
        for (let j = 0; j <= i; j = j + 1) {
            A[i][j] = j % 2 === 0 ? j : -j;
        }
    }
    return A;
}
```

A. $\Theta(1)$
B. $\Theta(\log n)$
C. $\Theta(n)$
D. $\Theta(n^2)$
E. $\Theta(n^3)$
F. $\Theta(n \log n)$
G. $\Theta(2^n)$
H. $\Theta(n^{1/2})$

(2) [2 marks] (MCQ)

What is the order of growth of the **time** needed to apply the above function `dat` to argument *n*?

A. $\Theta(1)$
B. $\Theta(\log n)$
C. $\Theta(n)$
D. $\Theta(n^2)$
E. $\Theta(n^3)$
F. $\Theta(n \log n)$
G. $\Theta(2^n)$
H. $\Theta(n^{1/2})$

(3) [3 marks] (MCQ)

What is the order of growth of the **time** needed to apply the following function `f` to argument *n*?

```
function f(n) {
    const h = x => x === 0 ? 0 : h(x - 1);
    const g = x => x <= 1 ? h(n) : g(x / 2);
    return g(n);
}
```

A. $\Theta(1)$
B. $\Theta(\log n)$
C. $\Theta(n)$
D. $\Theta(n^2)$
E. $\Theta(n^3)$
F. $\Theta(n \log n)$
G. $\Theta(2^n)$
H. $\Theta(n^{1/2})$

Section B: Recursion [7 marks]

(4) [3 marks] (MCQ)

What type of process does the function `r` give rise to for `n` > 1?

```
function r(n) {
    return n === 1 ? 1 : r(r(n - 1));
}
```

A. An iterative process

B. A recursive process

C. Not enough information to determine

(5) [4 marks]

Complete the declaration of function `power`, which takes as arguments a number `x` and a non-negative integer `n`, and returns the value x^n. For example,

```
power(2, 5);  //returns 32
power(3, 2);  //returns 9
power(8, 0);  //returns 1
```

Your function must not make use of the primitive function `math_pow`.

Write your answer only in the dashed-line box(es).

Section C: Higher Order Functions [9 marks]

(6) [4 marks]

Complete the declaration of function `app_count`, which takes as argument a unary function `f`, and returns a wrapper function. When the wrapper function is applied to an argument `x`, it displays the number of times the wrapper function has been called and returns the value of `f(x)`. Note, this function does not memoize the value of `f(x)`. For example,

```
const s = app_count(math_sqrt);
s(4);  // displays 1, returns 2
s(4);  // displays 2, returns 2
s(25); // displays 3, returns 5
```

Write your answer only in the dashed-line box(es).

(7) [5 marks]

Complete the declaration of function `fancy_sum`, which takes as an argument a list of integers, and returns the sum of the squares of the odd numbers in that list. For example,

```
fancy_sum(enum_list(1, 3)); // returns 1² + 3² = 10
fancy_sum(enum_list(0, 4)); // returns 1² + 3² = 10
fancy_sum(list(2, 1, 3, 1, 4, 5)); // returns 1² + 3² + 1² + 5² = 36
```

You may want to refer to the list processing functions in the Appendix.

Write your answer only in the dashed-line box(es).

Section D: Skip-Connected Lists [17 marks]

A ***Skip-Connected List* (*SCL*)** is either `null` or a pair whose head is an integer or a ***skip-connection***, and whose tail is an SCL. A *skip-connection* of a pair *p* points to a pair *q* in the SCL in the tail of *p*, that is, any pair *q* pointed to by the *skip-connection* of pair *p* is also reachable from *p* using a sequence of one or more `tail` operations on *p*.

An example SCL is shown in the box-and-pointer diagram below.

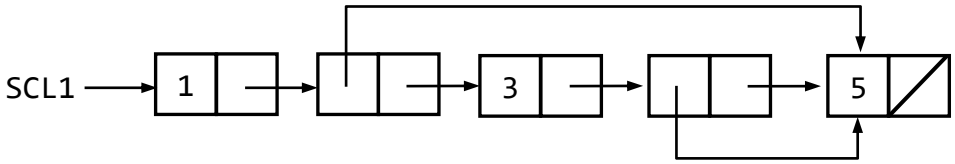


Diagram 1

(8) [4 marks]

Complete the given Source program such that after its evaluation, `S` will be an SCL that has the same structure as `SCL1` in Diagram 1.

Note: You may call the `tail` function at most 10 times.

Write your answer only in the dashed-line box(es).

(9) [3 marks]

Complete the declaration of function `flatten_SCL`, which takes as argument an SCL `S`, and returns a list that contains only all the data elements of the input SCL, and their order in the result list is the same as that in the input SCL. For example, using the SCL in Diagram 1 as input, `flatten_SCL(SCL1)` should return `list(1, 3, 5)`.

Write your answer only in the dashed-line box(es).

(10) [5 marks]

Complete the declaration of function `skip_to`, which takes as arguments an SCL `S` and a positive integer `idx`, and modifies the head of `S` to become a skip-connection that points to the pair at position `idx` in the list `S`. You may assume that `length(S)` ≥ 2 and `idx` < `length(S)`.

Example 1:

```
const SCL2 = list(1, 2, 3, 4);
skip_to(SCL2, 1); // result is shown in Diagram 2
```

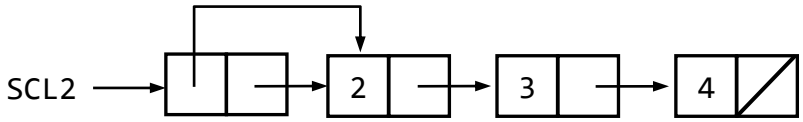


Diagram 2

Example 2:

```
const SCL3 = list(1, 2, 3, 4);
skip_to(SCL3, 3); // result is shown in Diagram 3
```

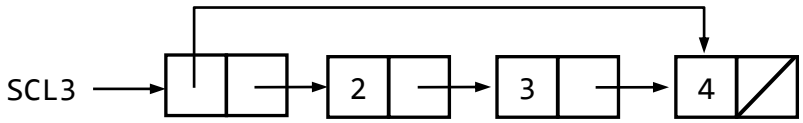


Diagram 3

Write your answer only in the dashed-line box(es).

(11) [5 marks]

Skip-connections in an SCL allow for faster traversals to the last pair of the SCL. We define the ***diameter*** of an SCL as the minimum number of pairs one must visit to reach the last pair from the first pair of the SCL. For example, `SCL1` in Diagram 1 has a diameter of **3**, `SCL2` in Diagram 2 has a diameter of **4**, and `SCL3` in Diagram 3 has a diameter of **2**.

Complete the declaration of function `diameter`, which takes as argument an SCL `S`, and returns its diameter. For example,

```
diameter(null);    // returns 0
diameter(list(8)); // returns 1
diameter(SCL1);    // returns 3
diameter(SCL2);    // returns 4
diameter(SCL3);    // returns 2
```

Hint: You may want to use the primitive function `math_min`, where `math_min(x, y)` returns the smaller of `x` and `y`.

Write your answer only in the dashed-line box(es).

Section E: Variations on a Theme [15 marks]

The questions in this section explore the relationship between iterations, functions, and variables.

(12) [3 marks] (MCQ)

What is the result of evaluating the following Source §3 program?

```
const a = [];
function f(i) {
    return i + 5;
}
let i = 0;
while (i < 10) {
    a[i] = f(i);
    i = i + 1;
}
a[4];
```

A. `5`
B. `9`
C. `10`
D. `11`
E. `14`
F. `15`
G. `16`
H. Error: `Cannot assign new value to constant a.`

(13) [3 marks] (MCQ)

What is the result of evaluating the following Source §3 program?

```
const a = [];
function f(i) {
    return x => i + x;
}
let i = 0;
while (i < 10) {
    a[i] = f(i);
    i = i + 1;
}
a[4](5);
```

A. `5`
B. `9`
C. `10`
D. `11`
E. `14`
F. `15`
G. `16`
H. Error: `Cannot assign new value to constant a.`

(14) [3 marks] (MCQ)

What is the result of evaluating the following Source §3 program?

```
const a = [];
let i = 0;
while (i < 10) {
    a[i] = x => i + x;
    i = i + 1;
}
a[4](5);
```

A. 5
B. 9
C. 10
D. 11
E. 14
F. 15
G. 16
H. Error: `Cannot assign new value to constant a.`

(15) [3 marks] (MCQ)

What is the result of evaluating the following Source §3 program?

```
const a = [];
function g(i) {
    if (i < 10) {
        a[i] = x => i + x;
        g(i + 1);
    }
}
g(0);
a[4](5);
```

A. 5
B. 9
C. 10
D. 11
E. 14
F. 15
G. 16
H. Error: `Cannot assign new value to constant a.`

(16) [3 marks] (MCQ)

What is the result of evaluating the following Source §3 program?

```
const a = [];
let i = 0;
function g() {
    if (i < 10) {
        a[i] = x => i + x;
        i = i + 1;
        g();
    }
}
g();
a[4](5);
```

A. 5

B. 9

C. 10

D. 11

E. 14

F. 15

G. 16

H. Error: `Cannot assign new value to constant a.`

Section F: Histograms [8 marks]

Assume given an array **A** and integers a_min and a_max, such that a_min ≤ a_max and such that every element of **A** is an integer within the range [a_min, a_max], the ***histogram*** of **A** from a_min to a_max is an array **H** of length a_max − a_min + 1, such that **H**[i] is the number of occurrences of the integer i + a_min in array **A**. For example, given

```
A = [5, 2, 2, 6, 8, 2, 5, 2, 5, 8]
```

the histogram of **A**, assuming a_min = 2 and a_max = 8, is

```
H = [4, 0, 0, 3, 1, 0, 2]
```

(17) [4 marks] (MCQ)

Complete the declaration of function make_histogram, which takes as arguments an array A and integers a_min and a_max, and returns the histogram of A from a_min to a_max. Note that a_min ≤ a_max and every element of A is an integer within the range [a_min, a_max].

```
function make_histogram(A, a_min, a_max) {
    const N = array_length(A);
    const H = [];
    const H_len = a_max - a_min + 1;

    for (let i = 0; i < H_len; i = i + 1) {
        H[i] = 0;
    }

    /* YOUR SOLUTION */

    return H;
}
```

Example:

```
const A = [5, 2, 2, 6, 8, 2, 5, 2, 5, 8];
make_histogram(A, 2, 8); // returns [4, 0, 0, 3, 1, 0, 2]
```

Which of the following is the correct answer for the part marked ***/* YOUR SOLUTION */***?

A.	`for (let k = 0; k < N; k = k + 1) { H[ A[k] - a_min ] = H[ A[k] - a_min ] + 1; }`
B.	`for (let k = 0; k < H_len; k = k + 1) { H[ A[k + a_min] ] = H[ A[k + a_min] ] + 1; }`
C.	`for (let k = a_min; k <= a_max; k = k + 1) { H[ A[k] ] = H[ A[k] ] + 1; }`
D.	`for (let k = 0; k < N; k = k + 1) { H[ A[k] ] = H[ A[k] ] + 1; }`
E.	`for (let k = 0; k < H_len; k = k + 1) { H[ A[k] ] = H[ A[k] ] + 1; }`
F.	`for (let k = 0; k < N; k = k + 1) { A[ H[k] ] = A[ H[k] ] + 1; }`
G.	`for (let k = 0; k < H_len; k = k + 1) { H[k] = H[k] + A[k]; }`

(18) [4 marks] (MCQ)

Complete the declaration of function `hisort`, which takes as arguments an array `A` and integers `a_min` and `a_max`, and returns a new array that has all the elements of array **`A`** sorted in ascending order. Note that `a_min` ≤ `a_max` and every element of `A` is an integer within the range [`a_min`, `a_max`].

```
function hisort(A, a_min, a_max) {
    const N = array_length(A);
    const H = make_histogram(A, a_min, a_max);
    const H_len = array_length(H);
    const B = [];
    let B_index = 0;

    for (let i = 0; i < H_len; i = i + 1) {
        for (let k = 0; k < H[i]; k = k + 1) {

            /* YOUR SOLUTION */

        }
    }
    return B;
}
```

We assume the `make_histogram` function from the preceding question is correctly implemented.

Example:

```
const A = [5, 2, 2, 6, 8, 2, 5, 2, 5, 8];
hisort(A, 2, 8); // returns [2, 2, 2, 2, 5, 5, 5, 6, 8, 8]
```

Which of the following is the correct answer for the part marked ***`/* YOUR SOLUTION */`***?

A.	`B[B_index] = k;` `B_index = B_index + 1;`
B.	`B[B_index] = k + a_min;` `B_index = B_index + 1;`
C.	`B[B_index] = k + a_min;` `B_index = B_index + H[i];`
D.	`B[B_index] = i;` `B_index = B_index + 1;`
E.	`B[B_index] = i + a_min;` `B_index = B_index + 1;`
F.	`B[B_index] = i + a_min;` `B_index = B_index + H[i];`
G.	`B[k] = i + a_min;`
H.	`B[i] = k + a_min;`

Section G: Streams [17 marks]

(19) [2 marks] (MCQ)

Consider the following program (refer to the Appendix for the implementation of `stream_ref`):

```
const rep0 = () => pair(0, rep0);
stream_ref(rep0(), 10);
```

How many **pairs** are created during the evaluation of the above program?

A. 0
B. 1
C. 2
D. 5
E. 8
F. 10
G. 11
H. 12

(20) [2 marks] (MCQ)

Consider the following program:

```
const rep0 = pair(0, () => rep0);
stream_ref(rep0, 10);
```

How many **pairs** are created during the evaluation of the above program?

A. 0
B. 1
C. 2
D. 5
E. 8
F. 10
G. 11
H. 12

(21) [2 marks] (MCQ)

Consider the following program:

```
const rep123 = pair(1,
                    () => pair(2,
                               () => pair(3, () => rep123)));
stream_ref(rep123, 10);
```

How many **pairs** are created during the evaluation of the above program?

A. 1
B. 3
C. 5
D. 7
E. 8
F. 10
G. 11
H. 12

(22) [3 marks] (MCQ)

Recall the function `memo_fun` from the lectures:

```
function memo_fun(fun) {
    let already_run = false;
    let result = undefined;

    function mfun() {
        if (!already_run) {
            result = fun();
            already_run = true;
            return result;
        } else {
            return result;
        }
    }
    return mfun;
}
```

Now, consider the following program:

```
const rep123 = pair(1,
                    memo_fun(() => pair(2,
                                        () => pair(3, () => rep123))));
stream_ref(rep123, 10);
```

How many **pairs** are created during the evaluation of the above program?

A. 1
B. 3
C. 5
D. 7
E. 8
F. 10
G. 11
H. 12

(23) [3 marks] (MCQ)

Consider the following program (where `memo_fun` is given in the preceding question):

```
const rep123 = pair(1,
                    () => pair(2,
                               memo_fun(() => pair(3, () => rep123))));
stream_ref(rep123, 10);
```

How many **pairs** are created during the evaluation of the above program?

A. 1
B. 3
C. 5
D. 7
E. 8
F. 10
G. 11
H. 12

(24) [5 marks]

Complete the declaration of function `repeat`, which takes as arguments a stream `S` and a positive integer `k`, and returns a stream that has every element of `S` occurring `k` consecutive times in the result stream. Note that the input stream `S` can be a finite or infinite stream.

Example:

```
const integers = integers_from(1);

eval_stream(repeat(integers, 3), 12);
// returns list(1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4)

eval_stream(repeat(integers, 1), 12);
// returns list(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)
```

Hint: Refer to the Appendix for the function `build_stream`.

Write your answer only in the dashed-line box(es).

Section H: Tombstone Diagrams [8 marks]

(25) [8 marks]

Recall that, to program Lego Mindstorms robots with Source, we made use of the following two components:

1. A **Source** to **SVML** compiler written in **JavaScript**
2. An **SVML** interpreter written in **EV3** machine code

With Mindstorms robots being discontinued by Lego, our Avengers have created new robot kits that use the **ARM6** processor. To facilitate the transition to the new robots, the Avengers have also implemented an **EV3** to **ARM6** compiler written in **JavaScript**.

You have the **Controller** program that has already been compiled into **SVML**. You want to run the **Controller** program on a new **ARM6** robot, with the help of the interpreters and compilers given above. You have access to an **x86** computer, which has a web browser that can interpret **JavaScript** programs.

Complete the given tombstone diagrams (T-diagrams) to demonstrate how you would run the **Controller** program on the **ARM6** robot.

Section I: CSE Machine in Alternate Universes [11 marks]

For your reference, here are the functions that implement environments, assignment, and name lookup in the CSE machine.

```
function enclosing_environment(env) { return tail(env); }
function first_frame(env) { return head(env); }
const the_empty_environment = null;
function make_frame(symbols, values) { return pair(symbols, values); }
function frame_symbols(frame) { return head(frame); }
function frame_values(frame) { return tail(frame); }
function extend_environment(ns, vs, e) {
    return pair(make_frame(ns, vs), e);
}
function lookup_symbol_value(symbol, env) {
    function env_loop(env) {
        function scan(symbols, vals) {
            return is_null(symbols)
                   ? env_loop(enclosing_environment(env))
                   : symbol === head(symbols)
                   ? head(vals)
                   : scan(tail(symbols), tail(vals));
        }
        if (env === the_empty_environment) {
            error(symbol, "unbound name");
        } else {
            const frame = first_frame(env);
            return scan(frame_symbols(frame), frame_values(frame));
        }
    }
    return env_loop(env);
}
function assign_symbol_value(symbol, val, env) {
    function env_loop(env) {
        function scan(symbols, vals) {
            return is_null(symbols)
                   ? env_loop(enclosing_environment(env))
                   : symbol === head(symbols)
                   ? set_head(vals, val)
                   : scan(tail(symbols), tail(vals));
        }
        if (env === the_empty_environment) {
            error(symbol, "unbound name -- assignment");
        } else {
            const frame = first_frame(env);
            return scan(frame_symbols(frame), frame_values(frame));
        }
    }
    return env_loop(env);
}
```

The questions in this section explore this version of the function `extend_environment` and alternative versions. We assume that all other functions remain unchanged.

(26) [3 marks] (MCQ)

Using the original version of the function `extend_environment`:

```
function extend_environment(ns, vs, e) {
    return pair(make_frame(ns, vs), e);
}
```

what will be displayed by the `display` function as a result of evaluating the following program in the CSE machine?

```
const x = 1;
const y = 2;
{
    const x = 10 + y;
    const y = 20;
    display(x);
}
```

A. `3`
B. `11`
C. `12`
D. `21`
E. `30`
F. Nothing: an error will occur

(27) [4 marks] (MCQ)

Using the following alternative version of the function `extend_environment`:

```
function extend_environment(ns, vs, e) {
    return list(is_null(e)
                ? make_frame(ns, vs)
                : make_frame(append(ns, frame_symbols(first_frame(e))),
                             append(vs, frame_values(first_frame(e)))));
}
```

what will be displayed by the `display` function as a result of evaluating the following program in the CSE machine?

```
const x = 1;
const y = 2;
{
    const x = 10 + y;
    const y = 20;
    display(x);
}
```

A. `3`
B. `11`
C. `12`
D. `21`
E. `30`
F. Nothing: an error will occur

(28) [4 marks] (MCQ)

Using the following alternative version of the function `extend_environment`:

```
function extend_environment(ns, vs, e) {
    return list(is_null(e)
                ? make_frame(ns, vs)
                : make_frame(append(frame_symbols(first_frame(e)), ns),
                             append(frame_values(first_frame(e)), vs)));
}
```

what will be displayed by the `display` function as a result of evaluating the following program in the CSE machine?

```
const x = 1;
const y = 2;
{
    const x = 10 + y;
    const y = 20;
    display(x);
}
```

A. `3`
B. `11`
C. `12`
D. `21`
E. `30`
F. Nothing: an error will occur

——— END OF QUESTIONS ———

Appendix

We assume the following pre-declared functions in Source §4 are declared as follows:

```
function map(f, xs) {
    return is_null(xs)
           ? xs
           : pair(f(head(xs)), map(f, tail(xs)));
}
function filter(pred, xs) {
    return is_null(xs)
           ? null
           : pred(head(xs))
           ? pair(head(xs), filter(pred, tail(xs)))
           : filter(pred, tail(xs));
}
function accumulate(f, initial, xs) {
    return is_null(xs)
           ? initial
           : f(head(xs), accumulate(f, initial, tail(xs)));
}
function append(xs, ys) {
    return is_null(xs)
           ? ys
           : pair(head(xs), append(tail(xs), ys));
}
```

```
function stream_tail(s) {
    return tail(s)();
}
function stream_map(f, s) {
    return is_null(s)
           ? null
           : pair(f(head(s)), () => stream_map(f, stream_tail(s)));
}
function stream_filter(p, s) {
    return is_null(s)
           ? null
           : p(head(s))
           ? pair(head(s), () => stream_filter(p, stream_tail(s)))
           : stream_filter(p, stream_tail(s));
}
function stream_append(xs, ys) {
    return is_null(xs)
           ? ys
           : pair(head(xs), () => stream_append(stream_tail(xs), ys));
}
function stream_ref(s, n) {
    return n === 0
           ? head(s)
           : stream_ref(stream_tail(s), n - 1);
}
function enum_stream(low, hi) {
    return low > hi
           ? null
           : pair(low, () => enum_stream(low + 1, hi));
}
function build_stream(fun, n) {
    function build(i) {
        return i >= n
               ? null
               : pair(fun(i), () => build(i + 1));
    }
    return build(0);
}
function eval_stream(s, n) {
    return n === 0
           ? null
           : pair(head(s), eval_stream(stream_tail(s), n - 1));
}
function integers_from(n) {
    return pair(n, () => integers_from(n + 1));
}
```

——— END OF PAPER ———

National University of Singapore

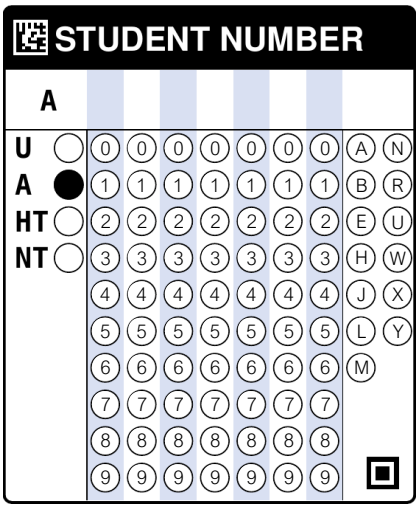
CS1101S — Programming Methodology

AY2023/2024 Semester 1

Final Assessment

Time allowed: 2 hours

ANSWER SHEET



(1) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(2) [2 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(3) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(4) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(5) [4 marks]

```
function power(x, n) {
    return 
          ? 
          : 
}
```

(6) [4 marks]

```
function app_count(f) {
    let count = 0;
    function wrapper(x) {
        
    }
    
}
```

(7) [5 marks]

```
function fancy_sum(xs) {
    return accumulate(
                [                                                    ],
                [                                                    ],
                map(  [                                              ],
                    filter(  [                                       ],
                             xs)));
}
```

(8) [4 marks]

```
const S = list(1, 2, 3, 4, 5);

set_head(  [                                                         ]);

set_head(  [                                                         ]);
```

(9) [3 marks]

```
function flatten_SCL(S) {
                  [                                                  ]
    return filter(                                                   );
}
```

(10) [5 marks]

```
function skip_to(S, idx) {
    function helper(scl, idx) {
        [                                                            ]
        [                                                            ]
        [                                                            ]
        [                                                            ]
    }
    set_head(  [                                                     ]);
}
```

(11) [5 marks]

```
function diameter(S) {
    if (is_null(S)) {
        [                                                    ]
    } else if ( ! is_pair(head(S)) ) {
        [                                                    ]
    } else {
        [                                                    ]
    }
}
```

(12) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(13) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(14) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(15) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(16) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(17) [4 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(18) [4 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(19) [2 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(20) [2 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(21) [2 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

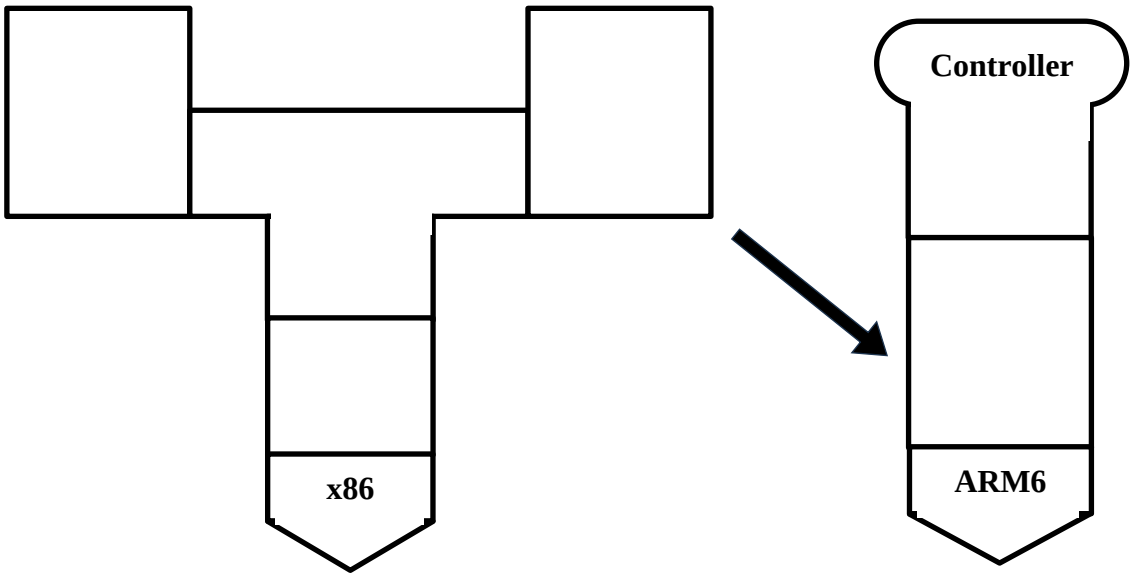
(22) [3 marks] ◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(23) [3 marks] ◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(24) [5 marks]

```
function repeat(S, k) {
  return is_null(S)
    ? null
    : stream_append(
        [                                                    ],
        [                                                    ]
      );
}
```

(25) [8 marks]



(26) [3 marks] ◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(27) [4 marks] ◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(28) [4 marks] ◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

END OF ANSWER SHEET